# 100Plante

Détection de maladies sur feuilles de plantes par transfert d'apprentissage

## Documentation technique

Dépôt du code : https://github.com/POUKONE/100Plante
API : https://100plante-backend.fly.dev
Application : https://100-plante.vercel.app
Date du document : 9 octobre 2026

# Table des matières

**1. Présentation et architecture . . . 3**
1.1 Objet du document . . . 3
1.2 Vue d'ensemble . . . 3
1.3 Flux d'une analyse . . . 4
1.4 Arborescence du dépôt . . . 4
**2. Prérequis et installation . . . 5**
2.1 Environnement ayant servi au développement . . . 5
2.2 Backend . . . 5
2.3 Frontend . . . 6
**3. Configuration . . . 6**
3.1 Variables d'environnement du backend . . . 6
3.2 Variable du frontend . . . 7
3.3 Configuration Fly.io . . . 7
**4. Exécution . . . 7**
4.1 Backend en local . . . 7
4.2 Frontend en local . . . 7
4.3 Backend en conteneur . . . 8
**5. Interface de programmation (API) . . . 8**
5.1 Généralités . . . 8
5.2 GET /api/health . . . 8
5.3 POST /api/predict . . . 8
5.4 Codes d'erreur . . . 9
**6. Modules du backend . . . 10**
6.1 Mode placeholder . . . 10
6.2 Prétraitement . . . 10
**7. Pipeline de données et d'entraînement . . . 11**
7.1 Enchaînement des scripts . . . 11
7.2 Données . . . 12
7.3 Modèle de transfert d'apprentissage . . . 12
7.4 Baseline . . . 13
7.5 Évaluation . . . 13
**8. Règles de décision et d'affichage . . . 14**
8.1 Côté API . . . 14
8.2 Côté interface . . . 14
**9. Frontend . . . 14**
**10. Tests et vérifications . . . 15**
**11. Déploiement et exploitation . . . 16**
11.1 Backend sur Fly.io . . . 16
11.2 Frontend sur Vercel . . . 16
11.3 Surveillance . . . 16
11.4 Remplacer le modèle . . . 16
**12. Limites connues et évolutions . . . 17**
12.1 Limites . . . 17
12.2 Évolutions possibles . . . 17
**Annexe A. Les 38 classes : effectifs et résultats par classe . . . 18**

# 1. Présentation et architecture

## 1.1 Objet du document

Ce document décrit le fonctionnement technique de 100Plante : un classifieur de maladies de feuilles (38 classes, 14 espèces, jeu de données PlantVillage), une API qui le sert et une interface web qui l'utilise. Il permet de réinstaller le projet, de le lancer, de le tester, de le déployer et de reprendre l'entraînement du modèle. Il s'appuie sur l'état du dépôt (https://github.com/POUKONE/100Plante) au 9 octobre 2026. Le contexte, les résultats et l'historique figurent dans le dossier final et dans le compte rendu chronologique.

## 1.2 Vue d'ensemble

Le système comporte deux chaînes distinctes. La chaîne hors ligne prépare les données, entraîne et évalue les modèles ; elle s'exécute en local, une seule fois, et produit un fichier de modèle. La chaîne en ligne se compose d'un frontend Next.js hébergé sur Vercel et d'une API FastAPI hébergée sur Fly.io dans un conteneur Docker ; le modèle est chargé en mémoire une fois, au démarrage de l'API.

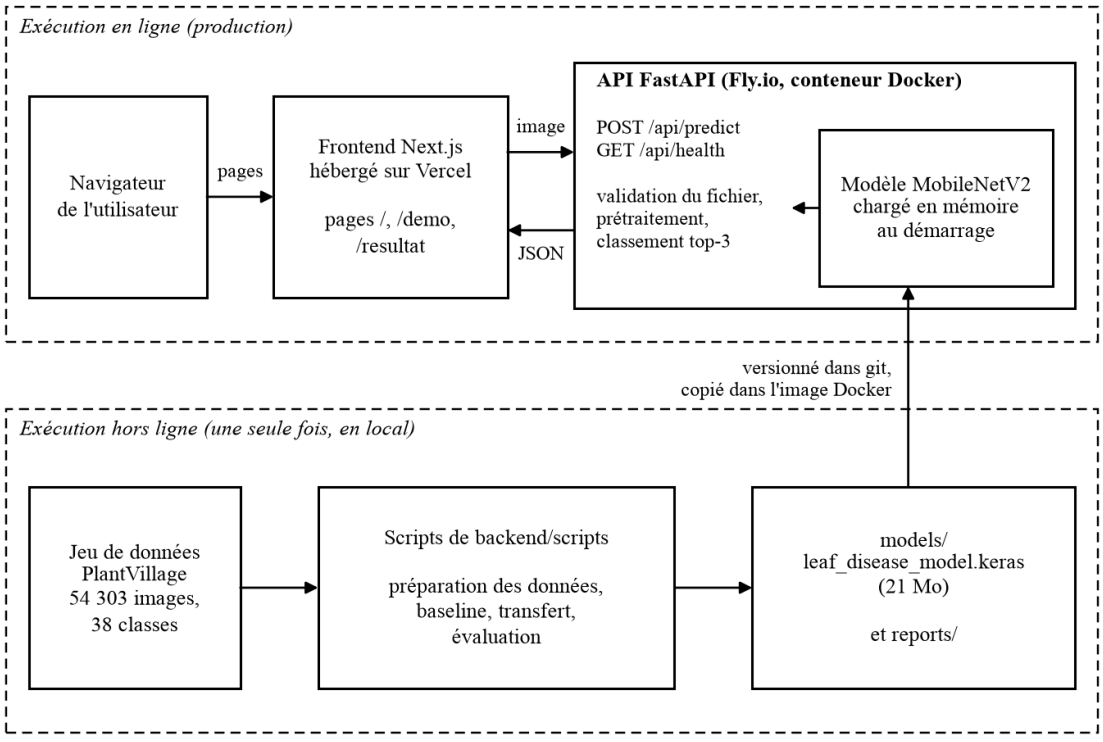



**Figure 1.** Architecture générale : exécution en ligne (navigateur, frontend, API) et exécution hors ligne (données, scripts, modèle).

## 1.3 Flux d'une analyse

1. L'utilisateur choisit une photo (JPEG ou PNG, 10 Mo au plus) sur la page `/demo`.
2. Le navigateur en produit une copie réduite (900 px au plus, JPEG) pour l'aperçu, puis envoie l'original en `multipart/form-data` à `POST /api/predict`.
3. L'API vérifie le type de contenu, lit le fichier, vérifie sa taille, décode l'image, la convertit en RVB et la redimensionne à 224 × 224 pixels.
4. Le modèle renvoie 38 probabilités ; l'API retient les trois plus élevées et indique si la première est inférieure au seuil de 0,70.
5. Le navigateur range la copie réduite et la réponse dans le `sessionStorage` puis ouvre `/resultat`, qui affiche le diagnostic, la fiche descriptive de la maladie et la fiabilité propre à la classe.

## 1.4 Arborescence du dépôt

Le fichier du modèle (`backend/models/leaf_disease_model.keras`, 21 Mo) est versionné pour que l'image Docker puisse l'embarquer sans étape de téléchargement. Les données (`backend/data`) et les autres fichiers de `backend/models` ne le sont pas.

```
100Plante/
  README.md, render.yaml, .gitignore
  docs/cahier-des-charges.md
  backend/
    app/
      main.py                  point d'entrée FastAPI
      core/config.py           configuration (pydantic-settings)
      api/routes/health.py     GET /api/health
      api/routes/predict.py    POST /api/predict
      ml/labels.py             les 38 classes, dans l'ordre du modèle
      ml/preprocess.py         décodage et redimensionnement
      ml/model.py              chargement, préchauffage, inférence
      schemas/prediction.py    modèles Pydantic des réponses
    scripts/                   pipeline de données et d'entraînement
    tests/test_predict.py      tests de l'API (pytest)
    reports/                   rapport d'évaluation et métriques
    models/                    modèle entraîné
    Dockerfile, fly.toml, .env.example, requirements*.txt
  frontend/
    app/                       pages (/, /demo, /resultat), styles, polices
    components/                BrandMark, HowItWorks, NeuropathTrail, ...
    lib/                       api, image, resultStore, diseaseInfo, ...
    package.json, next.config.ts, eslint.config.mjs
```

# 2. Prérequis et installation

## 2.1 Environnement ayant servi au développement

**Tableau 1.** Versions des outils relevées sur le poste de développement.

| Élément | Valeur relevée |
|---|---|
| Poste | Mac à puce Apple M1, 8 Go de mémoire, macOS (noyau Darwin 25.6.0) |
| Python | 3.11.6, géré avec pyenv (fichier `backend/.python-version`) |
| Node.js et npm | 24.11.0 et 11.6.1 |
| Docker | 28.5.1 |
| Fly.io (flyctl) | 0.4.77 |

Aucune autre version n'a été essayée. La version 3.11 est compatible avec TensorFlow 2.18.0 (Python 3.9 à 3.12) ; l'image Docker utilise la même version mineure (`python:3.11-slim`).

## 2.2 Backend

Les dépendances sont réparties en quatre fichiers, afin que l'image de production n'embarque que le nécessaire à l'API.

**Tableau 2.** Fichiers de dépendances du backend.

| Fichier | Contenu | Utilisé pour |
|---|---|---|
| `requirements.txt` | fastapi 0.115.6, uvicorn 0.34.0, python-multipart 0.0.20, pydantic-settings 2.7.1, tensorflow 2.18.0, pillow 11.1.0, numpy 1.26.4, scikit-learn 1.6.1, pytest 8.3.4, httpx 0.28.1 | API, tests, image Docker |
| `requirements-data.txt` | tensorflow-datasets 4.9.10, tensorflow-metadata 1.16.1 | Téléchargement de PlantVillage |
| `requirements-baseline.txt` | scikit-image 0.25.0 | Extraction HOG de la baseline |
| `requirements-eval.txt` | matplotlib 3.10.0 | Matrice de confusion |

`tensorflow-metadata` est volontairement épinglé à 1.16.1 : la version 1.21.0 embarque des fichiers protobuf générés avec une version 6 de protobuf, alors que TensorFlow 2.18.0 impose une version inférieure à 6.

```
cd backend
python -m venv .venv
./.venv/bin/pip install -r requirements.txt
```

## 2.3 Frontend

Le frontend n'a que trois dépendances d'exécution : next 16.2.12, react 19.2.4 et react-dom 19.2.4. TypeScript 5, ESLint 9 et eslint-config-next 16.2.12 servent au développement.

```
cd frontend
npm install
```

# 3. Configuration

## 3.1 Variables d'environnement du backend

La configuration est centralisée dans la classe `Settings` (`backend/app/core/config.py`, bibliothèque pydantic-settings). Chaque champ peut être surchargé par une variable d'environnement du même nom en majuscules ou par un fichier `.env` placé dans le dossier d'exécution ; le fichier `.env.example` en donne un modèle.

**Tableau 3.** Paramètres du backend (`Settings`).

| Variable | Valeur par défaut | Rôle |
|---|---|---|
| `APP_NAME` | `100Plante API` | Titre de l'API dans la documentation OpenAPI |
| `API_PREFIX` | `/api` | Préfixe de toutes les routes |
| `MODEL_PATH` | `backend/models/leaf_disease_model.keras` | Fichier du modèle ; s'il n'existe pas, l'API démarre en mode placeholder |
| `IMAGE_SIZE` | `(224, 224)` | Taille d'entrée du modèle |
| `CONFIDENCE_THRESHOLD` | `0.70` | Sous ce seuil, `low_confidence` vaut vrai |
| `TOP_K` | `3` | Nombre de classes renvoyées |
| `MAX_UPLOAD_SIZE_BYTES` | `10485760` (10 Mo) | Taille maximale du fichier reçu |
| `ALLOWED_CONTENT_TYPES` | `image/jpeg`, `image/png` | Types acceptés |
| `CORS_ALLOW_ORIGINS` | `http://localhost:3000`, `http://localhost:5173` | Origines autorisées, séparées par des virgules |

Deux points demandent de l'attention. Premièrement, `CORS_ALLOW_ORIGINS` est un champ de type chaîne, découpé sur les virgules par une propriété (`cors_allow_origins_list`). Un type `tuple` aurait fait décoder la variable en JSON avant toute validation, et une simple liste séparée par des virgules aurait empêché l'API de démarrer : c'est l'incident qui a bloqué le premier déploiement sur Fly.io. Deuxièmement, `IMAGE_SIZE` et `ALLOWED_CONTENT_TYPES` sont typés `tuple` : par le même mécanisme, leur surcharge par variable d'environnement exigerait une valeur JSON. Ce cas n'a pas été testé, aucune surcharge de ces deux champs n'étant utilisée.

## 3.2 Variable du frontend

`NEXT_PUBLIC_API_URL` donne l'adresse de l'API. Sans valeur, le frontend utilise `http://localhost:8000`. La variable est lue à la construction de l'application (préfixe `NEXT_PUBLIC_`) : sur Vercel, un changement impose donc un nouveau déploiement. En local, elle se place dans `frontend/.env.local`, qui n'est pas versionné.

## 3.3 Configuration Fly.io

**Tableau 4.** Configuration de déploiement du backend (`backend/fly.toml`).

| **Clé de** `fly.toml` | **Valeur** | **Rôle** |
|---|---|---|
| `app`, `primary_region` | `100plante-backend`, `cdg` | Nom de l'application et région (Paris) |
| `env.PORT`, `internal_port` | `8000` | Port d'écoute du conteneur |
| `force_https` | `true` | Redirection HTTP vers HTTPS |
| `auto_stop_machines`, `min_machines_running` | `false`, `1` | Machine toujours allumée, pour éviter le redémarrage du modèle |
| `checks` | `GET /api/health`, toutes les 15 s, délai de grâce 30 s | Contrôle de santé |
| `vm` | 1 Go de mémoire, 1 processeur partagé | Dimensionnement de la machine |

Le fichier `render.yaml` à la racine décrit un déploiement équivalent sur Render. Il n'a jamais été exécuté : le quota gratuit de Render étant atteint, l'hébergement s'est fait sur Fly.io.

# 4. Exécution

## 4.1 Backend en local

```
cd backend
./.venv/bin/uvicorn app.main:app --reload --port 8000
```

L'API est alors disponible sur `http://localhost:8000` et sa documentation interactive (générée par FastAPI) sur `http://localhost:8000/docs`. Au démarrage, le modèle est chargé puis préchauffé par une prédiction à vide ; le journal l'indique (« Modèle chargé… », « Warmup du modèle effectué »). Si le port 8000 est déjà occupé, Uvicorn s'arrête avec « Address already in use » : il faut alors arrêter le processus en cours ou changer de port.

## 4.2 Frontend en local

```
cd frontend
npm run dev
```

L'interface est disponible sur `http://localhost:3000`. Les commandes `npm run build`, `npm run start` et `npm run lint` construisent, servent et analysent le code.

### 4.3 Backend en conteneur

```
cd backend
docker build -t 100plante-backend .
docker run -p 8000:8000 100plante-backend
```

L'image part de `python:3.11-slim`, installe `libgomp1` (requis par TensorFlow), puis les dépendances de `requirements.txt` avant de copier le code et le modèle ; cet ordre permet de réutiliser la couche des dépendances tant que `requirements.txt` ne change pas. Le serveur écoute sur le port de la variable `PORT`, 8000 à défaut. Le fichier `.dockerignore` exclut données, scripts, tests et rapports de l'image.

## 5. Interface de programmation (API)

### 5.1 Généralités

Toutes les routes sont préfixées par `/api`. L'API est en production à l'adresse https://100plante-backend.fly.dev. Les origines autorisées par CORS se limitent aux méthodes `GET` et `POST`. Aucune authentification n'est en place.

### 5.2 GET /api/health

Renvoie l'état du service. `model_loaded` est l'inverse de `is_placeholder`. En production, la réponse attendue est `status` égal à `ok`, `model_loaded` à vrai, `is_placeholder` à faux et `num_classes` à 38.

```
{
  "status": "ok",
  "model_loaded": true,
  "is_placeholder": false,
  "num_classes": 38
}
```

### 5.3 POST /api/predict

Reçoit une image dans le champ `file` d'un formulaire `multipart/form-data`.

```
curl -X POST http://localhost:8000/api/predict \
     -F "file=@feuille.jpg;type=image/jpeg"
```

La réponse a la forme suivante. Les valeurs de l'exemple sont purement illustratives : elles ne proviennent pas d'un appel réel.

```
{
  "predicted": {
    "label": "Tomato___Late_blight",
    "species": "Tomato",
    "condition": "Late blight",
    "confidence": 0.91
  },
  "top_k": [ {...}, {...}, {...} ],
  "low_confidence": false,
  "inference_time_ms": 120.0,
  "is_placeholder": false
```

```
}
```

**Tableau 5.** Champs de la réponse de `POST /api/predict`.

| Champ | Type | Signification |
|---|---|---|
| `predicted` | objet | Classe la plus probable (identique au premier élément de `top_k`) |
| `label` | chaîne | Identifiant brut de la classe, de la forme `Espece___Etat` |
| `species`, `condition` | chaînes | Espèce et état déduits du libellé (tirets bas remplacés par des espaces) |
| `confidence` | nombre entre 0 et 1 | Probabilité attribuée par le modèle |
| `top_k` | liste de 3 objets | Trois classes les plus probables, par confiance décroissante |
| `low_confidence` | booléen | Vrai si la confiance de `predicted` est inférieure à `CONFIDENCE_THRESHOLD` |
| `inference_time_ms` | nombre | Durée du décodage, du prétraitement et de la prédiction côté serveur |
| `is_placeholder` | booléen | Vrai si aucun modèle entraîné n'est chargé (résultat factice) |

## 5.4 Codes d'erreur

**Tableau 6.** Erreurs renvoyées par `POST /api/predict`.

| Code | Cas | Message (champ `detail`) |
|---|---|---|
| 415 | Type de contenu autre que JPEG ou PNG (déclaré par le client) | Type de fichier non supporté : … Attendu : JPEG ou PNG. |
| 413 | Fichier de plus de 10 Mo, constaté après lecture complète | Fichier trop volumineux (max 10 Mo). |
| 400 | Octets reçus non décodables comme image | Impossible de décoder l'image envoyée |
| 422 | Champ `file` absent | Réponse standard de FastAPI (comportement non testé ici) |

Le contrôle du type repose sur l'en-tête déclaré par le client, non sur le contenu : un fichier non image déclaré `image/jpeg` est arrêté plus loin, par le décodage (code 400). Le contrôle de taille intervient après lecture complète du fichier en mémoire ; il ne protège donc pas contre un envoi très volumineux.

# 6. Modules du backend

**Tableau 7.** Modules de `backend/app`.

| Module | Rôle | Points notables |
|---|---|---|
| `app/main.py` | Crée l'application FastAPI, branche CORS et les routes | Fonction `lifespan` : le modèle est chargé une seule fois et rangé dans `app.state.model` |
| `app/core/config.py` | Classe `Settings`, fonction `get_settings()` | Mise en cache par `lru_cache` ; lecture de `.env` |
| `app/api/routes/health.py` | Route de santé | Lit l'état du modèle dans `app.state` |
| `app/api/routes/predict.py` | Route de prédiction | Valide, délègue au modèle, construit la réponse |
| `app/schemas/prediction.py` | Modèles Pydantic `ClassPrediction`, `PredictionResponse`, `HealthResponse` | Valident la forme des réponses et alimentent la documentation OpenAPI |
| `app/ml/labels.py` | Tuple de 38 libellés et objets `ClassInfo` | L'ordre doit être celui des dossiers triés par ordre alphabétique, donc celui du modèle |
| `app/ml/preprocess.py` | `load_image`, `to_model_input` | Partagé entre l'entraînement et l'inférence ; ne normalise pas les pixels |
| `app/ml/model.py` | Classe `LeafDiseaseModel` : `load`, `predict` | Import différé de TensorFlow ; préchauffage ; mode placeholder |

## 6.1 Mode placeholder

Si le fichier désigné par `MODEL_PATH` n'existe pas, l'API démarre quand même et répond avec une distribution de probabilités factice, calculée de façon déterministe à partir d'un hachage SHA-256 de l'image (une même image donne toujours le même résultat). Chaque réponse porte alors `is_placeholder` à vrai. Ce mode a servi à développer l'interface avant la fin de l'entraînement. Il ne doit jamais être présenté comme un diagnostic ; en production, `/api/health` permet de vérifier qu'il n'est pas actif.

## 6.2 Prétraitement

`load_image` décode les octets avec Pillow et convertit l'image en RVB (une erreur de décodage lève `InvalidImageError`, traduite en code 400). `to_model_input` redimensionne en 224 × 224 par interpolation bilinéaire, sans conserver le rapport d'aspect, et renvoie un tableau de forme (1, 224, 224, 3) en valeurs de 0 à 255. La normalisation en [−1, 1] n'est pas faite ici : elle est la première couche du modèle (`Rescaling`). L'appliquer aussi dans l'API produisait une double normalisation et des prédictions fausses (voir le dossier final, tableau des difficultés).

# 7. Pipeline de données et d'entraînement

## 7.1 Enchaînement des scripts

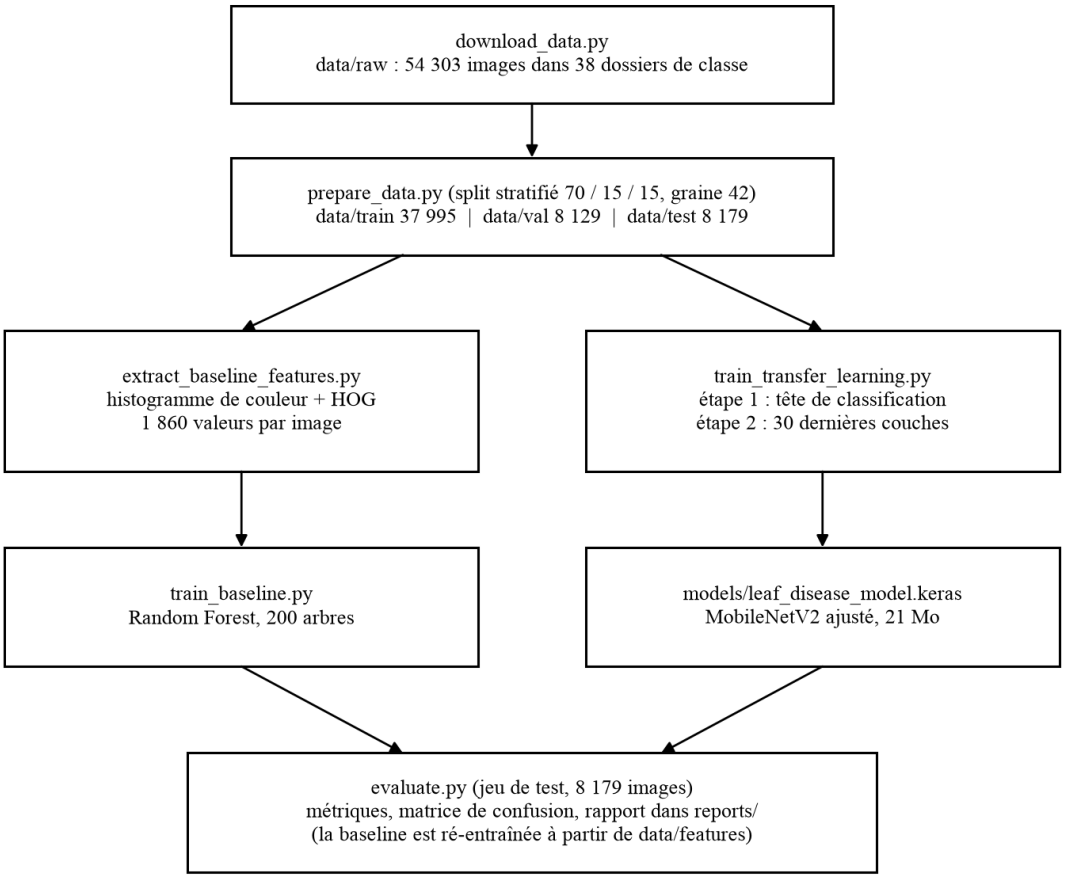



**Figure 2.** Enchaînement des scripts de `backend/scripts` et fichiers produits.

**Tableau 8.** Scripts du pipeline, à lancer depuis `backend` dans l'ordre du tableau.

| Script | Entrées | Sorties | Options |
|---|---|---|---|
| `download_data.py` | Jeu `plant_village` de TensorFlow Datasets | `data/raw/<classe>/*.jpg` : 54 303 images | `--output-dir` |
| `prepare_data.py` | `data/raw` | `data/train`, `data/val`, `data/test` | `--raw-dir`, `--output-dir` |
| `extract_baseline_features.py` | `data/train`, `val`, `test` | `data/features/*_features.npy`, `*_labels.npy`, `*_classes.txt` | `--split` (`train`, `val` ou `test`) |
| `train_baseline.py` | `data/features` | `models/baseline/random_forest_metrics.json` (et le modèle sérialisé) | `--output-dir`, `--model` (`random_forest` ou `svm`) |

| Script | Entrées | Sorties | Options |
|---|---|---|---|
| train_transfer_learning.py | data/train, data/val | models/leaf_disease_model.keras, points de contrôle, journaux TensorBoard | --data-dir, --model-path, --base-model, --epochs-head (8), --epochs-finetune (8), --fine-tune-layers (30) |
| evaluate.py | data/test, modèle, data/features | Dossier reports/ | --data-dir, --model-path, --report-dir |

```
python scripts/download_data.py
python scripts/prepare_data.py
python scripts/extract_baseline_features.py
python scripts/train_baseline.py
python scripts/train_transfer_learning.py
python scripts/evaluate.py
```

Les scripts de données importent le paquet `app` (pour la liste des classes) ; ils ajoutent donc le dossier `backend` au chemin d'import. Le dossier `data/raw` n'existe plus dans le poste de développement (supprimé pour libérer 1,6 Go) : reprendre le pipeline depuis le début suppose de relancer `download_data.py`.

## 7.2 Données

Le jeu PlantVillage compte 54 303 images réparties en 38 classes. `prepare_data.py` les répartit classe par classe, après tirage aléatoire de graine 42, en 70 % pour l'entraînement (37 995 images), 15 % pour la validation (8 129) et 15 % pour le test (8 179). Les fichiers sont copiés sans modification. Le déséquilibre est marqué : la plus petite classe compte 106 images d'entraînement (`Potato___healthy`), la plus grande 3 854 (`Orange___Haunglongbing_(Citrus_greening)`), soit un rapport d'environ 36. L'annexe A donne les effectifs de chaque classe.

## 7.3 Modèle de transfert d'apprentissage

**Tableau 9.** Architecture et paramètres du modèle.

| Élément | Choix |
|---|---|
| Base | MobileNetV2 pré-entraîné sur ImageNet, sans sa couche de classification (`include_top=False`) |
| Entrée | Images 224 × 224 × 3, pixels de 0 à 255 |
| Augmentation (entraînement seulement) | Retournement horizontal, rotation (0,15), zoom (0,1), luminosité (0,1) |
| Normalisation | Couche `Rescaling(1/127.5, offset=-1)` : valeurs dans [−1, 1] |
| Tête | Moyenne globale (`GlobalAveragePooling2D`), `Dropout(0.2)`, `Dense(38, softmax)` |
| Taille | 2 306 662 paramètres ; fichier de 21 Mo |

| Élément | Choix |
|---|---|
| Graine | 42 (`set_random_seed`) |
| Lots | 32 images |
| Perte | Entropie croisée catégorielle |
| Pondération des classes | `class_weight` calculé par `compute_class_weight('balanced')` sur le jeu d'entraînement |
| Métriques suivies | Exactitude et F1 macro (`F1Score(average='macro')`) |
| Rappels (callbacks) | `EarlyStopping` (surveille `val_loss`, patience 5, restauration des meilleurs poids), `ModelCheckpoint`, `TensorBoard` |

L'entraînement se déroule en deux étapes. À l'étape 1, la base est gelée et seule la tête est entraînée pendant 8 epochs avec l'optimiseur Adam au taux 1e-3. À l'étape 2, les 30 dernières couches de la base sont dégelées et l'ensemble est entraîné pendant 8 epochs au taux 1e-5. L'arrêt anticipé ne s'est pas déclenché : les 16 epochs ont été effectués. Le script refuse de continuer si l'ordre des classes lu sur le disque diffère de `app/ml/labels.py`, ce qui empêcherait le service d'afficher de mauvais libellés. L'option `--base-model efficientnetb0` existe mais lève une erreur `NotImplementedError` : ce modèle attend une autre normalisation, qui devrait être modifiée à la fois dans le script et dans l'API.

## 7.4 Baseline

La baseline sert de référence. Chaque image est réduite à 128 × 128 ; son vecteur de 1 860 valeurs concatène un histogramme de couleur (32 intervalles par canal, soit 96 valeurs, normalisé) et un histogramme de gradients orientés calculé sur l'image en niveaux de gris (9 orientations, cellules de 16 × 16 pixels, blocs de 2 × 2 cellules). Le classifieur est une forêt aléatoire de 200 arbres, avec pondération `balanced` et graine 42. Une machine à vecteurs de support (noyau RBF, C = 10) est disponible par l'option `--model svm` ; elle n'a pas été utilisée.

## 7.5 Évaluation

`evaluate.py` est le seul script qui lit le jeu de test. Il évalue le modèle de transfert, ré-entraîne la baseline à partir des caractéristiques déjà extraites (le fichier sérialisé a été supprimé pour libérer de l'espace ; l'opération prend environ deux minutes et est déterministe) et évalue celle-ci sur le même jeu. Il écrit dans `backend/reports` : `evaluation_report.md` (critères d'acceptation, comparaison, quinze confusions principales), `metrics.json`, `classification_report_transfer_learning.txt`, `confusion_matrix.csv` et `confusion_matrix.png`.

**Tableau 10.** Résultats de `metrics.json`.

| Mesure sur le jeu de test (8 179 images) | Transfert d'apprentissage | Baseline |
|---|---|---|
| Exactitude | 96,47 % | 80,95 % |
| F1 macro | 95,66 % | 73,58 % |

# 8. Règles de décision et d'affichage

## 8.1 Côté API

- Le modèle classe les 38 probabilités par ordre décroissant ; l'API en renvoie `TOP_K` (3).
- `low_confidence` est vrai lorsque la probabilité de la première classe est inférieure à `CONFIDENCE_THRESHOLD` (0,70). Le seuil n'est pas un rejet : une réponse est toujours renvoyée.
- Le modèle répartit toujours sa probabilité entre les 38 classes : une image qui n'est pas une feuille reçoit quand même une classe. Aucune détection d'image hors sujet n'est implémentée.

## 8.2 Côté interface

- **Fiabilité de la classe.** `frontend/lib/modelReliability.ts` contient la précision, le rappel et le F1 de chaque classe sur le jeu de test, recopiés du rapport de classification. Le niveau est « excellente » à partir d'une précision de 0,97, « bonne » à partir de 0,90, « moyenne » en dessous.
- **Confusions connues.** Pour certaines classes, une table codée à la main (tirée de la matrice de confusion) liste les classes avec lesquelles le modèle se trompe le plus ; elles sont signalées lorsqu'elles figurent dans les trois propositions.
- **Fiches descriptives.** `lib/diseaseInfo.ts` donne, pour chaque maladie, l'agent pathogène, les symptômes et le mode de propagation. Aucune recommandation de traitement n'y figure, conformément à la section 3.2 du cahier des charges.
- **Traductions.** `lib/translations.ts` fournit les noms français des espèces et des classes.

Ces trois tables sont écrites en dur. Après un nouvel entraînement ou un changement de classes, elles doivent être mises à jour à la main.

# 9. Frontend

**Tableau 11.** Organisation du frontend.

| Page ou module | Rôle |
|---|---|
| / (`app/page.tsx`) | Accueil : présentation, galerie des 14 espèces, section « Comment ça marche » |
| `/demo` | Choix d'une photo (appareil photo ou fichier, JPEG ou PNG, 10 Mo au plus), envoi à l'API, gestion des erreurs |
| `/resultat` | Diagnostic : classe, confiance, trois propositions, fiabilité, fiche de la maladie |
| `lib/api.ts` | Fonction `predictImage` ; classe `ApiError` ; adresse de l'API |
| `lib/image.ts` | `downscaleImage` : réduction de l'image à 900 px au plus, en JPEG qualité 0,85 |

| Page ou module | Rôle |
|---|---|
| `lib/resultStore.ts` | Stockage du dernier résultat dans le `sessionStorage` (clé `leaf-disease-detection:last-result`) et hook `useStoredResult` (`useSyncExternalStore`) |
| `lib/usePrefersReducedMotion.ts` | Respect de la préférence système de réduction des animations |
| `components/` | `BrandMark` (logo), `HowItWorks`, `NeuropathTrail` (fil Import, Analyse, Résultat), `SpecimenDeck` |

Next.js 16.2.12 est utilisé avec le routeur App Router, TypeScript et des CSS Modules ; les variables de couleur et de typographie sont dans `app/globals.css`. La police IBM Plex est auto-hébergée (`next/font/local`). Les points de rupture d'affichage ont été contrôlés à 375, 768 et 1 280 px de large. Le résultat transite par le `sessionStorage` du navigateur : il disparaît à la fermeture de l'onglet et n'est jamais envoyé à un serveur autre que l'API d'analyse.

# 10. Tests et vérifications

**Tableau 12.** Vérifications effectuées.

| Vérification | Portée | Résultat et date |
|---|---|---|
| `pytest` : `test_health_reports_consistent_model_state` | Santé : `status`, 38 classes, `model_loaded` inverse de `is_placeholder` | 5 tests sur 5 réussis, 9 octobre 2026 |
| `test_predict_returns_top_k_consistent_with_health` | Trois propositions, tri décroissant, confiances dans [0, 1] | idem |
| `test_predict_same_image_gives_same_result` | Déterminisme d'une même image | idem |
| `test_predict_rejects_unsupported_content_type` | Code 415 | idem |
| `test_predict_rejects_corrupt_image` | Code 400 | idem |
| ESLint et vérification des types TypeScript | Code du frontend | Aucune erreur, 9 octobre 2026 |
| Parcours complet en production | Image d'exemple, appel de l'API, page de résultat | Sans erreur, 3 août 2026 |
| Contrôle de santé de production | `GET /api/health` | Réponse attendue, 9 octobre 2026 |
| Contrôle de réponses avec le vrai modèle | Quatre appels dont deux sur des classes rares, après la correction de la double normalisation | Résultats jugés cohérents (contrôle manuel, non automatisé) |

Les tests utilisent `TestClient` de FastAPI dans un gestionnaire de contexte (`with TestClient(app)`), ce qui déclenche le chargement du modèle. Ils ont volontairement été écrits pour passer avec ou sans modèle entraîné : ils comparent des états entre eux (par exemple `model_loaded` et `is_placeholder`) et ne supposent pas une valeur. Ils emploient

une image synthétique unie ; ils ne vérifient donc pas la qualité de la classification, mesurée par `evaluate.py`. Aucun test automatisé n'existe pour le frontend ni pour les scripts d'entraînement.

# 11. Déploiement et exploitation

## 11.1 Backend sur Fly.io

```
cd backend
fly deploy
fly status
fly logs
fly secrets set CORS_ALLOW_ORIGINS="https://100-plante.vercel.app"
fly scale count 1
```

`fly deploy` construit l'image à partir du `Dockerfile` et applique `fly.toml`. `fly secrets set` définit la variable d'environnement des origines autorisées ; `fly scale count 1` limite le service à une machine (Fly.io en crée deux par défaut). Il faut inclure dans `CORS_ALLOW_ORIGINS` chaque adresse depuis laquelle le frontend est servi : l'adresse de production de Vercel est https://100-plante.vercel.app, mais les adresses de déploiements de prévisualisation sont différentes et ne sont autorisées que si elles sont listées. La valeur exacte actuellement enregistrée chez Fly.io n'a pas été relue pour la rédaction de ce document.

## 11.2 Frontend sur Vercel

Le dépôt GitHub est importé dans Vercel avec le dossier racine (`Root Directory`) réglé sur `frontend` et la variable `NEXT_PUBLIC_API_URL` égale à l'adresse de l'API. Les déploiements sont déclenchés par les envois sur le dépôt ; un `.vercelignore` empêche l'envoi de fichiers locaux sensibles.

## 11.3 Surveillance

Le contrôle de santé de Fly.io interroge `/api/health` toutes les quinze secondes. Aucun outil de supervision, d'alerte ou de mesure de charge supplémentaire n'est installé. Une panne est visible côté interface par le message « Impossible de joindre le serveur d'analyse ».

## 11.4 Remplacer le modèle

1. Reconstituer les données : `download_data.py` puis `prepare_data.py` (les dossiers `train`, `val` et `test` actuels peuvent être conservés si le découpage ne change pas).
2. Lancer `train_transfer_learning.py`, puis `evaluate.py` ; lire `reports/evaluation_report.md`.
3. Si les classes ou leur ordre changent : mettre à jour `app/ml/labels.py`, `lib/translations.ts`, `lib/diseaseInfo.ts` et `lib/modelReliability.ts`.
4. Mettre à jour dans `modelReliability.ts` les valeurs de précision, de rappel, de F1 et les confusions principales, et le nombre d'images de test.

5. Exécuter `pytest`, construire et tester l'image Docker, puis lancer `fly deploy` et vérifier `/api/health`.

# 12. Limites connues et évolutions

## 12.1 Limites

- **Démarrage lent.** Le chargement du modèle et son préchauffage allongent le démarrage de l'API. Le premier appel réel est mesuré à 262 ms après préchauffage ; la durée du démarrage lui-même n'a pas été chronométrée.
- **Mesures de temps.** Le temps d'inférence de l'API (52 à 262 ms) ne comprend pas le transfert réseau ; le temps de bout en bout n'a pas été chronométré.
- **Contrôle de taille.** La taille maximale est vérifiée après lecture complète du fichier en mémoire.
- **Redimensionnement.** L'entraînement (TensorFlow) et l'inférence (Pillow) redimensionnent tous deux en bilinéaire, mais les deux implémentations ne sont pas strictement identiques ; l'écart n'a pas été quantifié.
- **Données de référence en dur.** Fiches, traductions et fiabilités sont écrites dans le code du frontend.
- **Dépendances du frontend.** `npm audit` signale trois vulnérabilités de niveau « high » dans des dépendances internes de Next.js (postcss, sharp), non traitées : la correction automatique proposée imposerait une version majeure très ancienne de Next.js.
- **Déploiement Render.** Le fichier `render.yaml` n'a jamais été exécuté.
- **Domaine de validité.** Le modèle a été évalué sur des images de laboratoire uniquement.

## 12.2 Évolutions possibles

- Mesurer le temps de bout en bout et conserver les journaux de latence.
- Lire le fichier par blocs pour appliquer la limite de taille sans le charger entièrement.
- Ajouter une détection d'images hors sujet.
- Générer les traductions, fiabilités et fiches à partir de fichiers de données uniques plutôt que de les recopier à la main.
- Ajouter des tests de l'interface et une intégration continue qui exécute `pytest`, ESLint et la vérification des types.
- Convertir le modèle en TensorFlow Lite pour un usage mobile et vérifier sa taille.

# Annexe A. Les 38 classes : effectifs et résultats par classe

L'indice est la position de la classe dans `app/ml/labels.py`, qui est aussi celle des lignes et des colonnes de la matrice de confusion (figure 2 du dossier final). Les effectifs sont ceux des dossiers `backend/data/train`, `val` et `test`. La précision, le rappel et le F1 viennent de `reports/classification_report_transfer_learning.txt` (jeu de test) ; l'effectif de test de chaque classe y est identique à celui relevé sur disque.

**Tableau 13.** Effectifs par jeu et résultats par classe du modèle de transfert d'apprentissage.

| Indice | Classe | Train | Val | Test | Précision | Rappel | F1 |
|---|---|---|---|---|---|---|---|
| 0 | Apple___Apple_scab | 441 | 94 | 95 | 0,92 | 0,99 | 0,95 |
| 1 | Apple___Black_rot | 434 | 93 | 94 | 0,97 | 1,00 | 0,98 |
| 2 | Apple___Cedar_apple_rust | 192 | 41 | 42 | 1,00 | 0,98 | 0,99 |
| 3 | Apple___healthy | 1 151 | 246 | 248 | 1,00 | 1,00 | 1,00 |
| 4 | Blueberry___healthy | 1 051 | 225 | 226 | 1,00 | 1,00 | 1,00 |
| 5 | Cherry___Powdery_mildew | 736 | 157 | 159 | 1,00 | 0,97 | 0,99 |
| 6 | Cherry___healthy | 597 | 128 | 129 | 1,00 | 0,99 | 1,00 |
| 7 | Corn___Cercospora_leaf_spot Gray_leaf_spot | 359 | 76 | 78 | 0,82 | 0,83 | 0,83 |
| 8 | Corn___Common_rust | 834 | 178 | 180 | 0,98 | 0,98 | 0,98 |
| 9 | Corn___Northern_Leaf_Blight | 689 | 147 | 149 | 0,91 | 0,92 | 0,91 |
| 10 | Corn___healthy | 813 | 174 | 175 | 0,99 | 1,00 | 1,00 |
| 11 | Grape___Black_rot | 826 | 177 | 177 | 0,98 | 0,97 | 0,97 |
| 12 | Grape___Esca_(Black_Measles) | 968 | 207 | 208 | 0,97 | 0,99 | 0,98 |
| 13 | Grape___Leaf_blight_(Isariopsis_Leaf_Spot) | 753 | 161 | 162 | 0,98 | 1,00 | 0,99 |
| 14 | Grape___healthy | 296 | 63 | 64 | 1,00 | 1,00 | 1,00 |
| 15 | Orange___Haunglongbing_(Citrus_greening) | 3 854 | 826 | 827 | 1,00 | 1,00 | 1,00 |
| 16 | Peach___Bacterial_spot | 1 607 | 344 | 346 | 1,00 | 0,99 | 0,99 |
| 17 | Peach___healthy | 251 | 54 | 55 | 1,00 | 1,00 | 1,00 |
| 18 | Pepper,_bell___Bacterial_spot | 697 | 149 | 151 | 0,99 | 0,97 | 0,98 |
| 19 | Pepper,_bell___healthy | 1 033 | 221 | 223 | 0,96 | 0,99 | 0,98 |
| 20 | Potato___Early_blight | 700 | 150 | 150 | 0,98 | 0,99 | 0,98 |
| 21 | Potato___Late_blight | 700 | 150 | 150 | 0,96 | 0,92 | 0,94 |
| 22 | Potato___healthy | 106 | 22 | 24 | 0,81 | 0,92 | 0,86 |
| 23 | Raspberry___healthy | 259 | 55 | 57 | 0,98 | 1,00 | 0,99 |
| 24 | Soybean___healthy | 3 563 | 763 | 764 | 0,99 | 0,99 | 0,99 |

| Indice | Classe | Train | Val | Test | Précision | Rappel | F1 |
|---|---|---|---|---|---|---|---|
| 25 | Squash___Powdery_mildew | 1 284 | 275 | 276 | 0,99 | 1,00 | 1,00 |
| 26 | Strawberry___Leaf_scorch | 776 | 166 | 167 | 1,00 | 0,99 | 0,99 |
| 27 | Strawberry___healthy | 319 | 68 | 69 | 1,00 | 1,00 | 1,00 |
| 28 | Tomato___Bacterial_spot | 1 488 | 319 | 320 | 0,90 | 0,96 | 0,93 |
| 29 | Tomato___Early_blight | 700 | 150 | 150 | 0,92 | 0,73 | 0,81 |
| 30 | Tomato___Late_blight | 1 335 | 286 | 287 | 0,96 | 0,92 | 0,94 |
| 31 | Tomato___Leaf_Mold | 666 | 142 | 144 | 0,90 | 0,95 | 0,93 |
| 32 | Tomato___Septoria_leaf_spot | 1 239 | 265 | 267 | 0,94 | 0,92 | 0,93 |
| 33 | Tomato___Spider_mites Two-spotted_spider_mite | 1 173 | 251 | 252 | 0,92 | 0,81 | 0,86 |
| 34 | Tomato___Target_Spot | 982 | 210 | 212 | 0,76 | 0,90 | 0,82 |
| 35 | Tomato___Tomato_Yellow_Leaf_Curl_Virus | 3 749 | 803 | 805 | 1,00 | 0,97 | 0,98 |
| 36 | Tomato___Tomato_mosaic_virus | 261 | 55 | 57 | 0,98 | 0,89 | 0,94 |
| 37 | Tomato___healthy | 1 113 | 238 | 240 | 0,87 | 0,99 | 0,93 |
| | **Total** | 37 995 | 8 129 | 8 179 | | | |